

Reviewer Pack — Minimal Rank of Generalized Hypergeometric Series vs Resolut...

Entry 7b3d3e51505e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 16:54:08 UTC

Minimal Rank of Generalized Hypergeometric Series vs Resolution Proof Length for k-CNF

Entry ID: 7b3d3e51505e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 16:54:08 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions and Special Functions **Field B** (complexity object): Resolution Proof Complexity for k-CNF

Statement:

For all k-CNF formulas, the resolution proof length is upper-bounded by a function of the minimal rank of the generalized hypergeometric series representing the formula's clause indicators. Specifically, if F is a k-CNF formula with m clauses, then its resolution proof length is $O(m^{\theta(r(F))})$, where $r(F)$ is the minimal rank of F represented as a generalized hypergeometric series.

Rationale (proposer's reasoning):

Generalized hypergeometric functions provide a powerful tool for encoding combinatorial problems. If such functions can be used to represent clauses in a k-CNF formula with low rank, it suggests an efficient way to manipulate these formulas, which might imply lower bounds on the resolution proof length.

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cd31624c3614a89a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For k-CNF formulas with $n \leq 40$, if the correlation coefficient between minimal rank and resolution proof length exceeds 0.5 over 30 random seeds, it supports the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" generalized hypergeometric series AND "resolution proof complexity" k-CNF - "hypergeometric functions" AND "resolution proof length" k-CNF - CNF resolution proof length proportional to minimal rank generalized hypergeometric series

Top relevant hits considered: - [<http://arxiv.org/abs/1904.03812v5>] A new approach to hypergeometric transformation formulas - [<http://arxiv.org/abs/2506.09382v1>] On the exterior power structure of the cohomology groups for the general hypergeometric integral - [<http://arxiv.org/abs/math/0603646v2>] Uniform convergence of hypergeometric series - [<http://arxiv.org/abs/1404.0381>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/1204.3399v2>] On a strange evaluation of the hypergeometric series by Gosper - [s2:ca44ef9be97a1c211295785ba4b0290db90a6bf] international workshop on functions in algebra and geometry 2nd-6th 2022

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=20.8s

5.1 Generated Python source

```
import random
from itertools import combinations, permutations
from fractions import Fraction

def generate_kcnf(n: int, k: int) -> list:
    variables = [f'x{i}' for i in range(1, n + 1)]
    clauses = []
    for _ in range(k):
        clause = random.sample(variables, random.randint(1, n))
        if random.choice([True, False]):
            clause = [f'~{v}' for v in clause]
        clauses.append(clause)
    return clauses

def hypergeometric_rank(clause: list) -> int:
    rank = 0
    seen = set()
    for literal in clause:
        var = literal[1:] if literal.startswith('~') else literal
        if var not in seen:
            seen.add(var)
            rank += 1
    return rank

def resolution_length(clauses: list) -> int:
    assignment = {}
```

```

variables = {var[1:] for var in set(lit for clause in clauses for lit in clause)}
stack = [clauses]

def dpll(formula, assignment, variables):
    if not formula:
        return True
    unassigned_var = next((v for v in variables if v not in assignment), None)
    if unassigned_var is None:
        return False

    true_branch = dpll([c for c in formula if any(lit.startswith(f'+{unassigned_var}') or lit
    false_branch = dpll([c for c in formula if any(lit.startswith(f'+{unassigned_var}') or lit

    return true_branch or false_branch

while stack:
    current_clause = stack.pop()
    if not current_clause:
        continue
    unit_clause = next((c for c in current_clause if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        var = literal[1:] if literal.startswith('-') else literal
        assignment[var] = literal.startswith('+')
        stack.extend([c for c in current_clause if var not in c])
    else:
        literals = [lit for clause in current_clause for lit in clause]
        pairs = list(permutations(literals, 2))
        for pair in pairs:
            if pair[0].startswith('-') and pair[1] == f'+{pair[0][1:]}':
                new_clause = [lit for lit in literals if lit not in pair]
                stack.append(new_clause)

return len(assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    k = 2 * n
    clauses = generate_kcnf(n, k)
    ranks = [hypergeometric_rank(clause) for clause in clauses]
    proof_lengths = [resolution_length(clauses) for _ in range(30)]

    if not all(ranks):
        return {
            "metric_name": "Hypergeometric Rank",
            "metric_value": sum(ranks) / len(ranks),
            "instances_tested": len(ranks),
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    correlation = 0
    n_tests = min(len(proof_lengths), len(ranks))
    for i in range(n_tests):
        correlation += (proof_lengths[i] - sum(proof_lengths) / n_tests) * (ranks[i] - sum(ranks) / n_tests)

```

```

correlation /= (n_tests * sum((x - sum(proof_lengths) / n_tests) ** 2 for x in proof_lengths)

return {
    "metric_name": "Hypergeometric Rank",
    "metric_value": correlation,
    "instances_tested": n_tests,
    "conjecture_holds": correlation > 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + [101, 103, 105]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std_dev = (sum((r['metric_value'] - mean) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        counterexample = min((r['counterexample'] for r in results if not r['conjecture_holds']),
                             first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_030d74a3.py", line 111, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_030d74a3.py", line 95, in run_trial

correlation /= (n_tests * sum((x - sum(proof_lengths) / n_tests) ** 2 for x in proof_lengths) * sum(

sum(ranks) / n_tests) ** 2 for y in ranks)) ** 0.5

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete successfully to provide a correlation coefficient and metric mean for the conjecture’s support condition.
| next: Re-run the test without errors to verify if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16454
2	preregistration	ollama_remote	glm4:latest	0	0	9620
3	novelty	ollama_remote	glm4:latest	0	0	8241
4	novelty	ollama_remote	glm4:latest	0	0	11321
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16510
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10870
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12901
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15529
9	judge	ollama_remote	glm4:latest	0	0	11745

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113190 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7b3d3e51505e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7b3d3e51505e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7b3d3e51505e.tar.gz (if generated)